$\text{Extracting the data from the CSV files from the JPL NASA Horizons application}$

```
import pandas as pd
import numpy as np
from pathlib import Path

data_dir = Path(
    "/home/suparnajit2005/GitHub/Classical_mechanics_simulations/"
    "Central-force-motion/Data/NASA"
)

Bodies = ["sun", "mercury", "venus", "earth", "mars", "jupiter"]

column_names_8 = [
    "JDTDB", "Calendar_Date",
    "X", "Y", "Z", "VX", "VY", "VZ"
]

column_names_11 = column_names_8 + ["LT", "RG", "RR"]

planet_data = {}

for body_name in Bodies:
    file_path = data_dir / f"horizons_results_{body_name}.csv"
    data_lines = []
    in_data_zone = False

    with open(file_path, "r") as f:
        for line in f:
            if "$$SOE" in line:
                in_data_zone = True
                continue

            if "$$EOE" in line:
                break

            if in_data_zone:
                cleaned_line = line.strip().rstrip(",")

                if cleaned_line:
                    data_lines.append(
                        [value.strip() for value in cleaned_line.split(",")]
                    )

    if not data_lines:
        raise ValueError(f"No data rows found in {file_path}")

    # Detect the actual number of columns in this file
    column_count = len(data_lines[0])

    if column_count == 8:
        columns = column_names_8
    elif column_count == 11:
        columns = column_names_11
    else:
        raise ValueError(
            f"{body_name}: expected 8 or 11 columns, "
            f"but found {column_count}. Example row: {data_lines[0]}"
        )

    # Verify all rows have the same number of fields
    bad_rows = [
        (i, len(row), row)
```

```
        for i, row in enumerate(data_lines)
        if len(row) != column_count
    ]

    if bad_rows:
        raise ValueError(
            f"{body_name}: inconsistent row format. "
            f"First problematic row: {bad_rows[0]}"
        )

    df = pd.DataFrame(data_lines, columns=columns)

    df["Calendar_Date"] = df["Calendar_Date"].str.strip()

    numeric_cols = [
        c for c in columns if c != "Calendar_Date"
    ]
    df[numeric_cols] = df[numeric_cols].apply(
        pd.to_numeric, errors="raise"
    )

    planet_data[body_name] = df

    print(
        f"{body_name}: {len(df):,} rows, "
        f"{len(df.columns)} columns"
    )
```

```
sun: 87,841 rows, 8 columns
mercury: 87,841 rows, 8 columns
venus: 87,841 rows, 8 columns
earth: 87,841 rows, 8 columns
mars: 87,841 rows, 8 columns
jupiter: 87,841 rows, 8 columns
```

```
# Confirm all bodies have the same timestamps
reference_times = planet_data["sun"]["JDTDB"].to_numpy()

for body_name in Bodies:
    times = planet_data[body_name]["JDTDB"].to_numpy()

    if not np.array_equal(times, reference_times):
        raise ValueError(
            f"Timestamp mismatch in {body_name}"
        )

positions = np.stack([
    planet_data[b][["X", "Y", "Z"]].to_numpy()
    for b in Bodies
])

velocities = np.stack([
    planet_data[b][["VX", "VY", "VZ"]].to_numpy()
    for b in Bodies
])

# Initial state at the first timestamp
state0 = np.concatenate([
    positions[:, 0, :].reshape(-1),
    velocities[:, 0, :].reshape(-1)
])
```

```
t_days = reference_times - reference_times[0]

print("Positions:", positions.shape)
print("Velocities:", velocities.shape)
print("Initial state:", state0.shape)
print("Elapsed time:", t_days[0], "to", t_days[-1], "days")
```

```
Positions: (6, 87841, 3)
Velocities: (6, 87841, 3)
Initial state: (36,)
Elapsed time: 0.0 to 61.0 days
```

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


#This class contains the initial position and velocity of the celestial body:

class body:
    def __init__(self,name,mass,pos,vel):
        self.name=name
        self.mass=mass
        self.pos=np.array(pos,dtype=float)
        self.vel=np.array(vel,dtype=float)



b1 = body("Sun", 1.0, np.array([0, 0, 0]), np.array([0, 0, 0]))

b2 = body("Mercury", 1.660e-7, np.array([0.387, 0, 0]), np.array([0, 0.0278, 0]))

b3 = body("Venus", 2.448e-6, np.array([0.723, 0, 0]), np.array([0, 0.0202, 0]))

b4 = body("Earth", 3.003e-6, np.array([1.000, 0, 0]), np.array([0, 0.0172, 0]))

b5 = body("Mars", 3.227e-7, np.array([1.524, 0, 0]), np.array([0, 0.0139, 0]))

b6 = body("Jupiter", 9.545e-4, np.array([5.203, 0, 0]), np.array([0, 0.00754, 0]))

b7 = body("Saturn", 2.858e-4, np.array([9.537, 0, 0]), np.array([0, 0.00557, 0]))

b8 = body("Uranus", 4.366e-5, np.array([19.191, 0, 0]), np.array([0, 0.00396, 0]))

b9 = body("Neptune", 5.151e-5, np.array([30.069, 0, 0]), np.array([0, 0.00313, 0]))


bodies=[b1,b2,b3,b4,b5,b6,b7,b8,b9]


N=len(bodies)

def acc(mass,pos):


    a=np.zeros((len(bodies),3))

    for i in range(len(bodies)):
        for j  in range(len(bodies)):
```

```
            if i!=j:
                G = 2.959122082855911e-4

                r=pos[i]-pos[j]

                ax=-((G*mass[j]*r[0])/(np.linalg.norm(r)**3))
                ay=-((G*mass[j]*r[1])/(np.linalg.norm(r)**3))
                az=-((G*mass[j]*r[2])/(np.linalg.norm(r)**3))

                a[i]+=np.array([ax,ay,az])

    return a



def eq_o_mot(t,state):
    N=len(bodies)
    pos=state[:3*N].reshape(N,3)
    vel=state[3*N:].reshape(N,3)

    m=np.array([])
    for i in range(N):
        m=np.append(m,bodies[i].mass)

    ac=acc(m,pos)

    return np.concatenate((vel.flatten(),ac.flatten()))


initial_pos=np.array([])
initial_vel=np.array([])

for i in range(N):
    initial_pos=np.append(initial_pos,bodies[i].pos)
    initial_vel=np.append(initial_vel,bodies[i].vel)

state0=np.concatenate((initial_pos,initial_vel))

t_span = (0, 365.25)

t_eval = np.linspace(0, 365.25, 1000)

sol=solve_ivp(eq_o_mot,t_span,state0,method="RK45",t_eval=t_eval)




import numpy as np
from pathlib import Path

data_dir = Path(
    "/home/suparnajit2005/GitHub/Classical_mechanics_simulations/"
    "Central-force-motion/Data/NASA"
)

names = ["Sun", "Mercury", "Venus", "Earth", "Mars", "Jupiter"]

def load_horizons(name):
    rows = []
```

```
    reading = False

    filepath = data_dir / f"horizons_results_{name.lower()}.csv"

    with open(filepath, "r") as file:
        for line in file:
            if "$$SOE" in line:
                reading = True
                continue

            if "$$EOE" in line:
                break

            if reading and line.strip():
                fields = [
                    x.strip()
                    for x in line.strip().rstrip(",").split(",")
                ]

                if len(fields) != 8:
                    raise ValueError(
                        f"{name}: expected 8 fields, got {len(fields)}"
                    )

                rows.append([
                    float(fields[0]),       # Julian Date
                    *map(float, fields[2:8])  # X,Y,Z,VX,VY,VZ
                ])

    return np.asarray(rows, dtype=float)


data = {name: load_horizons(name) for name in names}

# Keep the original Julian Dates
jd = data["Sun"][:, 0]

# Check that every body's timestamps match
for name in names:
    if not np.array_equal(data[name][:, 0], jd):
        raise ValueError(f"Timestamps do not match for {name}")

# Positions remain in km; velocities remain in km/s
positions = np.stack(
    [data[name][:, 1:4] for name in names], axis=1
)

velocities = np.stack(
    [data[name][:, 4:7] for name in names], axis=1
)

# Initial conditions at the first CSV timestamp
initial_pos = positions[0]
initial_vel = velocities[0]

# Elapsed time in seconds, suitable for solve_ivp
t_seconds = (jd - jd[0]) * 86400.0

state0 = np.concatenate([
    initial_pos.flatten(),
    initial_vel.flatten()
])
```

```
print("Positions:", positions.shape, "km")
print("Velocities:", velocities.shape, "km/s")
print("Initial state:", state0.shape)
print("Time:", t_seconds.shape, "seconds")
```

```
Positions: (87841, 6, 3) km
Velocities: (87841, 6, 3) km/s
Initial state: (36,)
Time: (87841,) seconds
```

```
sol
```